



浙江大学
ZHEJIANG UNIVERSITY

中国空间站机械臂运动仿真

《机器人技术与实践》实验报告

课程名称:	机器人技术与实践
小组成员:	金加康 吴必兴 沈学文 钱满亮 赵钰泓 项科深
组 号:	第三组
指导教师:	周春琳
报告日期:	2024 年 12 月 22 日

目录

1 实验内容与要求 3

1.1 实验背景 3

1.2 实验目的 3

1.3 实验要求 3

1.4 小组分工 3

2 机械臂运动学建模 3

2.1 机械臂结构分析 3

2.2 标准 DH 参数表 4

2.3 正运动学求解 4

3 逆运动学求解 5

3.1 问题描述 5

3.2 关节角 θ_1 、 θ_2 与 θ_6 的求解 5

3.3 关节角 θ_7 的求解 5

3.4 关节角 θ_3 、 θ_4 、 θ_5 的求解 6

3.5 多解处理 6

4 轨迹规划方法 6

4.1 五次多项式轨迹规划原理 6

4.2 边界条件与系数求解 7

4.3 分段轨迹规划策略 7

5 基座切换机制 7

5.1 切换原理 7

5.2 实现方法 8

5.3 关节角映射 8

6 代码实现 9

6.1 系统架构 9

6.2 轨迹规划器实现 9

6.3 逆运动学求解器实现 10

6.4 主控制程序实现 11

7 实验结果与分析 12

7.1 运动过程描述 12

7.2 速度连续性分析 12

8 实验总结 13

中国空间站机械臂运动仿真

金加康 吴必兴 沈学文 钱满亮 赵钰泓 项科深

§ 1 实验内容与要求

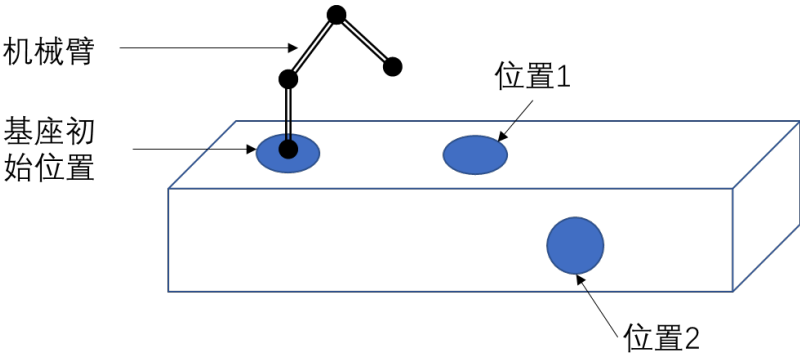
1.1 实验背景

中国空间站机械臂是我国自主研发的舱外大型空间机器人，具备舱段转位、航天员出舱活动支持、货物搬运等多种功能。其核心特征是采用对称式双末端执行器设计，两端均可作为基座或末端，使机械臂能够在空间站表面“行走”移动，极大地扩展了工作范围。本实验旨在通过仿真环境模拟这一独特的运动模式。

1.2 实验目的

- 1. 根据缩小版机械臂的结构与尺寸，利用标准 DH 方法建立正运动学模型。
- 2. 分析并推导该 7 自由度机械臂的逆运动学解析解。
- 3. 采用五次多项式轨迹规划方法，控制机械臂在空间站表面完成至少两步行走。
- 4. 实现基座切换机制，模拟机械臂“换手”行走的过程。
- 5. 借助 CoppeliaSim 仿真环境进行可视化演示，验证运动学模型与轨迹规划的正确性。

1.3 实验要求



- 1. 机械臂从初始位置移动至位置 1，再从位置 1 移动至位置 2，完成两步行走。
- 2. 运动过程中保证速度、加速度连续，避免运动突变。
- 3. 假设机械臂末端与基座之间接触后可通过电磁铁紧密吸合。
- 4. 提交完整的仿真代码、*.ttt 文件及实验报告。

1.4 小组分工

- 1. 机械臂运动学建模：金加康、吴必兴、赵钰泓、项科深
- 2. 代码撰写与仿真：吴必兴、沈学文、赵钰泓、钱满亮
- 3. 报告撰写：金加康、沈学文、吴必兴

§ 2 机械臂运动学建模

2.1 机械臂结构分析

本实验所用机械臂为中国空间站机械臂的缩小版仿真模型，采用 7 自由度串联结构。机械臂两端结构对称，分别标记为 A 端（左基座）和 B 端（右基座），中间通过关节 4 连接。机械臂的几何参数如下：

部件	参数	数值 (mm)
基座高度	d_1	120
肩部偏移	d_2	100
大臂长度	a_3	400
小臂长度	a_4	400
腕部偏移	d_6	100
末端高度	d_7	120

机械臂具有对称结构特性：当以 A 端为基座时，B 端为末端执行器；当以 B 端为基座时，A 端为末端执行器。这种对称设计使得机械臂可以通过切换基座实现“行走”运动。

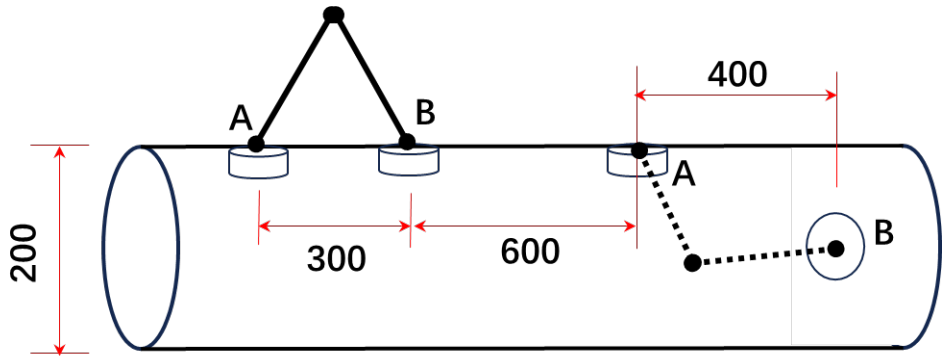


图 2 太空舱落足位置图

2.2 标准 DH 参数表

本实验采用**标准 DH 法**建立机械臂的运动学模型。以 A 端（左基座）为参考坐标系时，各连杆的 DH 参数如**表 1**所示。

表 1 以左基座为参考的标准 DH 参数表

关节 i	a_i (mm)	α_i (rad)	d_i (mm)	θ_i
1	0	$-\frac{\pi}{2}$	120	θ_1
2	0	$-\frac{\pi}{2}$	-100	θ_2
3	400	0	-150	θ_3
4	-400	0	-150	θ_4
5	0	$\frac{\pi}{2}$	0	θ_5
6	0	$\frac{\pi}{2}$	100	θ_6
7	0	0	120	θ_7

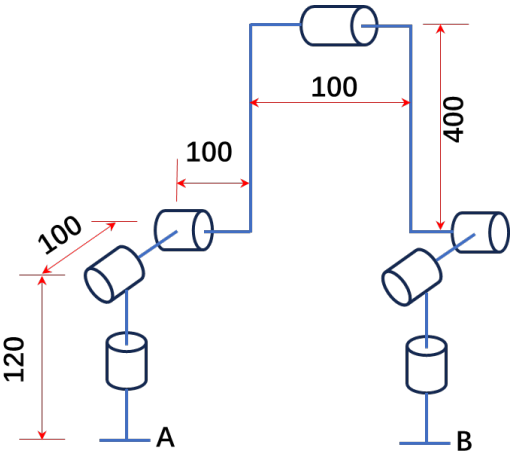


图 3 机械臂结构图

当以 B 端（右基座）为参考坐标系时，DH 参数表中的 d_2 和 d_6 取相反数，其余参数保持不变。这一特性为基座切换提供了理论基础。

2.3 正运动学求解

根据**标准 DH 法**，相邻坐标系之间的齐次变换矩阵为：

$${}^{i-1}_iT = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (1)$$

从基座到末端的总齐次变换矩阵由各相邻坐标系的齐次变换矩阵依次相乘得到:

$${}^0_7T = {}^0_1T {}^1_2T {}^2_3T {}^3_4T {}^4_5T {}^5_6T {}^6_7T \quad (2)$$

该变换矩阵包含了末端执行器相对于基座的位置向量 $[x, y, z]^T$ 和姿态旋转矩阵 R 。

§ 3 逆运动学求解

3.1 问题描述

逆运动学求解的目标是: 给定末端执行器的位姿 (位置和姿态), 求解各关节角度 $\theta_1, \theta_2, \dots, \theta_7$ 。由于本机械臂具有 7 个自由度而末端位姿仅有 6 个约束, 因此存在冗余自由度。本实验采用指定第一关节角 θ_1 的方式消除冗余, 将问题简化为 6 自由度逆运动学求解。

设末端位姿变换矩阵为:

$${}^0_7T = \begin{pmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (3)$$

3.2 关节角 θ_1 、 θ_2 与 θ_6 的求解

对位姿变换矩阵进行变换, 左乘 $({}^0_1T)^{-1}$ 和 $({}^1_2T)^{-1}$:

$$({}^1_2T)^{-1}({}^0_1T)^{-1}{}^0_7T = {}^2_3T {}^3_4T {}^4_5T {}^5_6T {}^6_7T = {}^2_7T \quad (4)$$

通过计算 2_7T 的第 (3,3) 和 (3,4) 元素, 可以建立关于 θ_2 和 θ_6 的方程:

$${}^2_7T(3,3) = -\sin \theta_6 = r_{13} \cos \theta_2 \sin \theta_1 - r_{23} \cos \theta_1 \cos \theta_2 - r_{33} \sin \theta_2 \quad (5)$$

$${}^2_7T(3,4) = -120 \sin \theta_6 - 300 = (120 - z) \sin \theta_2 + (x \sin \theta_1 - y \cos \theta_1) \cos \theta_2 \quad (6)$$

联立上述方程, 利用三角恒等式 $\sin^2 x + \cos^2 x = 1$, 可求得:

$$\theta_6 = -\arcsin({}^2_7T(3,3)) \text{ 或 } \pi + \arcsin({}^2_7T(3,3)) \quad (7)$$

$$\theta_2 = \text{atan2}(\sin \theta_1, \cos \theta_1), \quad \theta_2 = \text{atan2}(\sin \theta_2, \cos \theta_2) \quad (8)$$

3.3 关节角 θ_7 的求解

继续分析 2_7T 的元素:

$${}^2_7T(3,1) = -\cos \theta_6 \cos \theta_7 = r_{11} \cos \theta_2 \sin \theta_1 - r_{21} \cos \theta_1 \cos \theta_2 - r_{31} \sin \theta_2 \quad (9)$$

$${}^2_7T(3,2) = \cos \theta_6 \sin \theta_7 = r_{12} \cos \theta_2 \sin \theta_1 - r_{22} \cos \theta_1 \cos \theta_2 - r_{32} \sin \theta_2 \quad (10)$$

由于 θ_6 已知, $\cos \theta_6$ 为已知量, 故可求得:

$$\theta_7 = \text{atan2}\left(\frac{{}^2T(3,2)}{\cos\theta_6}, -\frac{{}^2T(3,1)}{\cos\theta_6}\right) \quad (11)$$

3.4 关节角 θ_3 、 θ_4 、 θ_5 的求解

通过 2T 的 (1,4)、(2,4)、(2,1)、(2,2) 元素, 可以建立关于 $\theta_3 + \theta_4 + \theta_5$ (记为 θ_{345}) 的方程。首先求解 θ_{345} :

$$m = -r_{11} \cos\theta_1 - r_{21} \sin\theta_1 \quad (12)$$

$$\theta_{345} = \arcsin\left(\frac{m}{\sqrt{a^2 + b^2}}\right) - \text{atan2}(a, b) \quad (13)$$

其中 $a = -\sin\theta_7$, $b = \cos\theta_7 \sin\theta_6$ 。

进一步, 通过几何关系求解 θ_3 和 θ_4 :

$$m_1 = \frac{120 \cos\theta_{345} \cos\theta_6 - 100 \sin\theta_{345} + (z - 120) \cos\theta_2 + x \sin\theta_1 \sin\theta_2 - y \cos\theta_1 \sin\theta_2}{400} \quad (14)$$

$$m_2 = \frac{120 \sin\theta_{345} \cos\theta_6 + 100 \cos\theta_{345} - x \cos\theta_1 - y \sin\theta_1 - 100}{400} \quad (15)$$

$$\theta_4 = \arccos\left(1 - \frac{m_1^2 + m_2^2}{2}\right) \quad (16)$$

最后:

$$\theta_3 = \text{atan2}\left(\frac{(1 + \cos\theta_4)m_1 + \sin\theta_4 m_2}{2 \sin\theta_4}, \frac{(1 - \cos\theta_4)m_1 - \sin\theta_4 m_2}{2(1 - \cos\theta_4)}\right) \quad (17)$$

$$\theta_5 = \theta_{345} - \theta_3 - \theta_4 \quad (18)$$

3.5 多解处理

由于逆运动学方程的非线性特性, 每组末端位姿最多存在 8 组可行解。在实际应用中, 需要根据以下准则选取最优解:

1. 关节限位约束: 剔除超出关节运动范围的解。
2. 连续性准则: 选择与当前关节状态变化最小的解, 保证运动连续性。
3. 奇异性规避: 避免选择接近奇异位形的解。

§ 4 轨迹规划方法

4.1 五次多项式轨迹规划原理

为保证机械臂运动过程中位置、速度、加速度的连续性, 本实验采用分段五次多项式进行关节空间轨迹规划。五次多项式具有 6 个待定系数, 恰好满足起点和终点的位置、速度、加速度共 6 个边界约束条件。

五次多项式的一般形式为:

$$\theta(t) = a_5 t^5 + a_4 t^4 + a_3 t^3 + a_2 t^2 + a_1 t + a_0 \quad (19)$$

相应的速度和加速度为:

$$\dot{\theta}(t) = 5a_5t^4 + 4a_4t^3 + 3a_3t^2 + 2a_2t + a_1 \quad (20)$$

$$\ddot{\theta}(t) = 20a_5t^3 + 12a_4t^2 + 6a_3t + 2a_2 \quad (21)$$

4.2 边界条件与系数求解

设轨迹段的起始时刻为 t_s ，结束时刻为 t_e ，边界条件为：

1. 起点位置 $\theta(t_s) = \theta_s$ ，终点位置 $\theta(t_e) = \theta_e$
2. 起点速度 $\dot{\theta}(t_s) = v_s$ ，终点速度 $\dot{\theta}(t_e) = v_e$
3. 起点加速度 $\ddot{\theta}(t_s) = a_s$ ，终点加速度 $\ddot{\theta}(t_e) = a_e$

将边界条件代入多项式方程，得到线性方程组：

$$\begin{pmatrix} t_s^5 & t_s^4 & t_s^3 & t_s^2 & t_s & 1 \\ 5t_s^4 & 4t_s^3 & 3t_s^2 & 2t_s & 1 & 0 \\ 20t_s^3 & 12t_s^2 & 6t_s & 2 & 0 & 0 \\ t_e^5 & t_e^4 & t_e^3 & t_e^2 & t_e & 1 \\ 5t_e^4 & 4t_e^3 & 3t_e^2 & 2t_e & 1 & 0 \\ 20t_e^3 & 12t_e^2 & 6t_e & 2 & 0 & 0 \end{pmatrix} \begin{pmatrix} a_5 \\ a_4 \\ a_3 \\ a_2 \\ a_1 \\ a_0 \end{pmatrix} = \begin{pmatrix} \theta_s \\ v_s \\ a_s \\ \theta_e \\ v_e \\ a_e \end{pmatrix} \quad (22)$$

通过求解上述线性方程组，即可得到五次多项式的系数 $a_0, a_1, \dots, a_5$ 。

4.3 分段轨迹规划策略

对于经过多个路径点的轨迹规划，采用分段五次多项式方法：

1. 路径点定义：指定 n 个关键路径点 $q_1, q_2, \dots, q_n$ 及其对应的时刻 $t_1, t_2, \dots, t_n$ 。
2. 中间点速度估算：采用中心差分法计算中间路径点的速度：

$$v_i = \frac{q_{i+1} - q_{i-1}}{t_{i+1} - t_{i-1}}, \quad i = 2, 3, \dots, n-1 \quad (23)$$

3. 中间点加速度估算：类似地，采用中心差分法计算加速度：

$$a_i = \frac{v_{i+1} - v_{i-1}}{t_{i+1} - t_{i-1}}, \quad i = 2, 3, \dots, n-1 \quad (24)$$

4. 边界条件：起点和终点的速度、加速度均设为零，保证机械臂从静止开始、静止结束：

$$v_1 = v_n = 0, \quad a_1 = a_n = 0 \quad (25)$$

5. 分段求解：对每相邻两个路径点之间的轨迹段，分别求解五次多项式系数。

这种分段规划方法保证了整个运动过程中关节角、角速度、角加速度的连续性，满足机械臂平稳运动的要求。

§ 5 基座切换机制

5.1 切换原理

中国空间站机械臂的独特之处在于其两端对称设计，均可作为基座或末端。机械臂在空间站表面“行走”时，需要周期性地切换基座：当一端固定于落足点时，另一端作为末端执行器移动；完成一步后，原末端变为新的基座，原基座变为新的末端。

在仿真环境中，基座切换的实现需要重新组织机械臂各部件的父子层级关系。以从左基座切换到右基座为例：

1. 原层级：左基座 → 左关节 1 → 左连杆 1 → ... → 关节 4 → ... → 右连杆 1 → 右关节 1 → 右基座
2. 新层级：右基座 → 右关节 1 → 右连杆 1 → ... → 关节 4 → ... → 左连杆 1 → 左关节 1 → 左基座

5.2 实现方法

基座切换通过 CoppeliaSim 的 `setObjectParent` 函数实现，该函数允许动态修改对象的父子关系。关键参数 `keep_in_place=True` 确保在修改层级关系时，各部件在世界坐标系中的位置保持不变。

从左基座切换到右基座的流程如下：

```

1  def change_base_LtoR(sim):
2      """从左基座切换到右基座为参考系"""
3      # 获取各部件句柄
4      l_base = sim.getObject('./L_Base')
5      r_base = sim.getObject('./R_Base')
6      # ... 获取其他部件句柄
7
8      # 重建层级关系：右臂为根节点
9      sim.setObjectParent(r_base, -1, 1) # 右基座设为根节点
10     sim.setObjectParent(r_joint1, r_base, 1)
11     sim.setObjectParent(r_link1, r_joint1, 1)
12     # ... 依次设置右臂各部件
13
14     # 连接中间关节
15     sim.setObjectParent(joint4, r_link3, 1)
16
17     # 连接左臂（反向）
18     sim.setObjectParent(l_link3, joint4, 1)
19     sim.setObjectParent(l_joint3, l_link3, 1)
20     # ... 依次设置左臂各部件（反向）
21     sim.setObjectParent(l_base, l_joint1, 1)

```

5.3 关节角映射

基座切换后，关节的物理意义发生变化。以右基座为参考时，原来的“左关节 1”变为“末端关节 7”，原来的“右关节 1”变为“基座关节 1”。因此，在设置关节位置时需要进行相应的映射：

```

1  def set_joint_positions(sim, joint_positions, base_type):
2      """根据基座类型设置关节位置"""
3      if base_type == 'B': # 右基座为参考系
4          sim.setJointPosition(r_joint1, joint_positions[0])
5          sim.setJointPosition(r_joint2, joint_positions[1])
6          sim.setJointPosition(r_joint3, joint_positions[2])
7          sim.setJointPosition(joint4, joint_positions[3])
8          sim.setJointPosition(l_joint3, joint_positions[4])
9          sim.setJointPosition(l_joint2, joint_positions[5])
10         sim.setJointPosition(l_joint1, joint_positions[6])
11     elif base_type == 'A': # 左基座为参考系
12         sim.setJointPosition(l_joint1, joint_positions[0])
13         # ... 类似映射

```


§ 6 代码实现

6.1 系统架构

本实验的仿真系统由四个核心模块组成：

1. **main.py**: 主控制程序，负责仿真流程控制、关节位置设置、速度监测与可视化。
2. **trajectory.py**: 轨迹规划模块，实现五次多项式轨迹生成与插值计算。
3. **base_switcher.py**: 基座切换模块，实现左右基座的动态切换。
4. **ik_solver.py**: 逆运动学求解器，提供 8 组解析解的计算。

6.2 轨迹规划器实现

Trajectory 类实现了分段五次多项式轨迹规划的完整功能：

```

1  class Trajectory:
2      """五次多项式轨迹规划器"""
3
4      def __init__(self, waypoints, time_points):
5          """初始化：给定路径点和时间点"""
6          self.waypoints = np.array(waypoints)
7          self.time_points = np.array(time_points)
8          self.num_waypoints = len(waypoints)
9          self._compute_waypoint_velocities_and_accelerations()
10         self._compute_polynomial_coefficients()
11
12     def _compute_waypoint_velocities_and_accelerations(self):
13         """计算路径点处的速度和加速度（中心差分法）"""
14         self.velocities = []
15         self.accelerations = []
16         for i in range(self.num_waypoints):
17             if i == 0 or i == self.num_waypoints - 1:
18                 velocity = np.array([0, 0, 0, 0, 0, 0, 0]) # 起终点速度为0
19             else:
20                 delta_q = self.waypoints[i + 1] - self.waypoints[i - 1]
21                 delta_t = self.time_points[i + 1] - self.time_points[i - 1]
22                 velocity = delta_q / delta_t
23                 self.velocities.append(velocity)
24         # 类似方法计算加速度...
25
26     def _compute_quintic_polynomial_segment(self, segment_idx):
27         """计算单段五次多项式系数"""
28         t_start = self.time_points[segment_idx]
29         t_end = self.time_points[segment_idx + 1]
30         # 构建系数矩阵并求解
31         coefficient_matrix = np.array([
32             [t_start**5, t_start**4, t_start**3, t_start**2, t_start, 1],
33             [5*t_start**4, 4*t_start**3, 3*t_start**2, 2*t_start, 1, 0],
34             [20*t_start**3, 12*t_start**2, 6*t_start, 2, 0, 0],

```

```

35         [t_end**5, t_end**4, t_end**3, t_end**2, t_end, 1],
36         [5*t_end**4, 4*t_end**3, 3*t_end**2, 2*t_end, 1, 0],
37         [20*t_end**3, 12*t_end**2, 6*t_end, 2, 0, 0],
38     ])
39     boundary_conditions = np.array([q_start, v_start, a_start,
40                                   q_end, v_end, a_end])
41     coefficients = np.linalg.solve(coefficient_matrix, boundary_conditions)
42     self.polynomial_coefficients.append(coefficients)
43
44     def GetCurvePosition(self, time):
45         """获取指定时刻的关节位置"""
46         for i in range(self.num_waypoints - 1):
47             if time < self.time_points[i + 1]:
48                 positions = []
49                 for joint_idx in range(7):
50                     coeffs = self.polynomial_coefficients[i, :, joint_idx]
51                     position = np.dot(coeffs,
52                                       [time**5, time**4, time**3, time**2, time, 1])
53                     positions.append(position)
54                 return positions
55         return self.waypoints[self.num_waypoints - 1]

```

6.3 逆运动学求解器实现

逆运动学求解器 ik_solver.py 实现了完整的解析求解流程:

```

1  def solve(target_transform, angle1, base_type='left'):
2      """求解逆运动学的8组可能解"""
3      x = target_transform[0, 3]
4      y = target_transform[1, 3]
5      z = target_transform[2, 3]
6      rotation_matrix = target_transform[0:3, 0:3]
7
8      # 计算关节2和关节6的候选解
9      a2_1, a2_2, a6_1, a6_2, a6_3, a6_4 = calc_angle2and6(
10         rotation_matrix, x, y, z, angle1)
11
12     # 计算关节7
13     a7_1 = calc_angle7(rotation_matrix, angle1, a2_1, a6_1)
14     a7_2 = calc_angle7(rotation_matrix, angle1, a2_2, a6_2)
15     # ...
16
17     # 计算关节3+4+5的角度和
18     a345_1 = calc_angle345(rotation_matrix, angle1, a6_1, a7_1, base_type)
19     # ...
20
21     # 计算关节3、4、5
22     a3_1, a4_1, a5_1, a3_5, a4_5, a5_5 = calc_angle34(
23         x, y, z, angle1, a2_1, a6_1, a345_1, base_type)

```

 Python

```

24     # ...
25
26     # 组合8组解
27     if base_type == 'right':
28         solutions = [
29             [angle1, -a2_1, a3_1, -a4_1, a5_1, -a6_1, a7_1],
30             # ... 其余7组解
31         ]
32     else:
33         solutions = [
34             [angle1, a2_1, a3_1, a4_1, a5_1, a6_1, a7_1],
35             # ... 其余7组解
36         ]
37     return solutions

```

6.4 主控制程序实现

主程序 main.py 协调各模块完成仿真流程：

```

1  # 仿真参数设置
2  SIMULATION_DURATION = 10 # 总仿真时长
3  PHASE_1_END = 5          # 第一阶段结束时间
4  PHASE_2_END = 6          # 第二阶段结束时间
5
6  # 预定义关键姿态
7  pose_initial = np.array([0, 0, 0, 0, 0, 0, 0])
8  pose_phase1_waypoint1 = np.array([0, 0.5, 0.5, 0, -0.172, -0.057, 0])
9  pose_phase1_waypoint2 = np.array([0, 1.9, 0.433, -1.540, -1.169, -1.047, 0])
10 pose_phase1_final = np.array([0, 2.094, 0.434, -1.540, -1.168, -1.047, 0])
11
12 # 主仿真循环
13 is_base_switched = False
14 while (current_time := sim.getSimulationTime()) < SIMULATION_DURATION:
15     # 第一阶段：从初始位置移动到目标位置（右基座）
16     if current_time < PHASE_1_END:
17         if not is_base_switched:
18             change_base_LtoR(sim)
19             is_base_switched = True
20             trajectory = Trajectory(
21                 [pose_initial, pose_phase1_waypoint1,
22                  pose_phase1_waypoint2, pose_phase1_final],
23                 [0, 1, 4, 5]
24             )
25             joint_positions = trajectory.GetCurvePosition(current_time)
26             set_joint_positions(sim, joint_positions, "B")
27
28     # 第二阶段：保持姿态等待吸合
29     elif current_time < PHASE_2_END:
30         set_joint_positions(sim, pose_phase1_final, "B")

```

```
31
32     # 第三阶段：返回最终姿态（左基座）
33     else:
34         if is_base_switched:
35             change_base_Rtol(sim)
36             is_base_switched = False
37         trajectory = Trajectory(
38             [pose_phase2_initial, pose_phase2_waypoint, pose_phase2_final],
39             [6, 9, 10]
40         )
41         joint_positions = trajectory.GetCurvePosition(current_time)
42         set_joint_positions(sim, joint_positions, "A")
43
44     client.step()
```

§ 7 实验结果与分析

7.1 运动过程描述

仿真实验成功实现了机械臂在空间站表面的两步行走运动，具体过程（视频见“第三组仿真结果.mp4”）如下：

- 1. **第一步（0-5 秒）**：以右端为基座，机械臂从初始直立姿态开始运动，左端末端执行器经过多个路径点的平滑轨迹，最终到达位置 1 的落足点。
- 2. **姿态保持（5-6 秒）**：机械臂保持姿态不变，模拟电磁铁吸合过程。此阶段为基座切换提供稳定的过渡期。
- 3. **第二步（6-10 秒）**：切换为左端基座，原右端变为末端执行器。机械臂继续运动，右端末端到达位置 2，完成第二步行走。

7.2 速度连续性分析

通过 CoppeliaSim 内置的 Graph 组件记录各关节的角速度变化曲线。观察曲线可以发现：

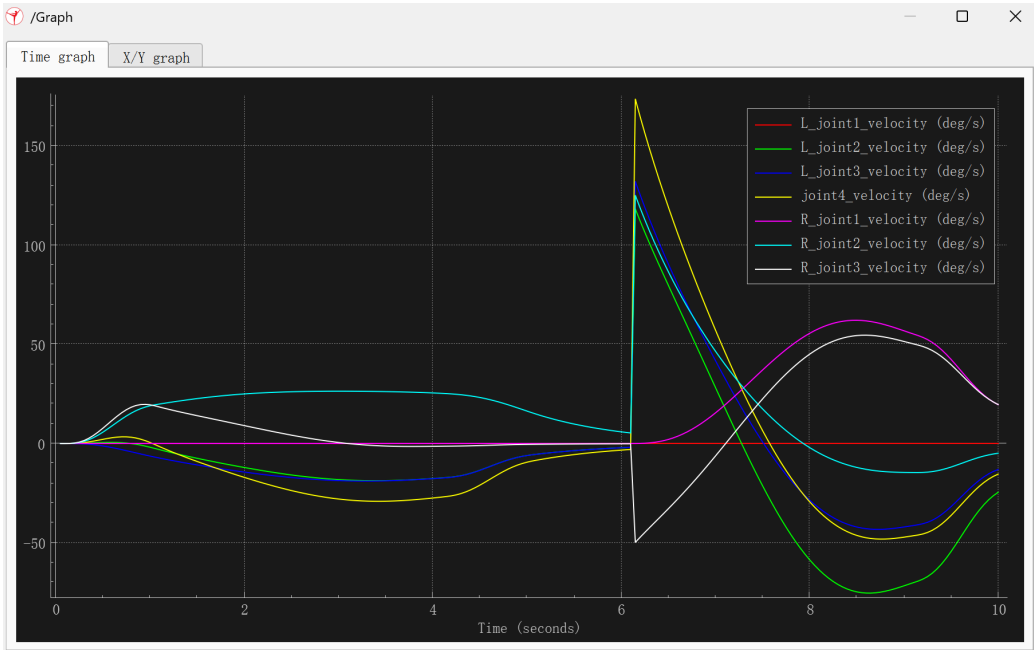


图 4 各关节的角速度变化曲线图

- 1. 在各轨迹段内，关节角速度呈现平滑的曲线形态，无尖锐拐点，表明加速度连续。
- 2. 在路径点处，速度值平滑过渡，没有出现跳变，证明分段五次多项式规划成功保证了速度连续性。
- 3. $t = 6$ 秒处速度曲线出现“突变”现象，这是由于基座切换导致关节编号重新映射，并非真实的速度不连续。从物理意义上看，每个实际关节的运动仍然是连续的。

§ 8 实验总结

本实验成功完成了中国空间站机械臂运动仿真的全部任务，主要成果包括：

- 1. **运动学建模：**建立了 7 自由度机械臂的标准 DH 参数模型，推导了完整的正逆运动学解析表达式。逆运动学求解器能够针对给定末端位姿输出 8 组可行解，为轨迹规划提供了数学基础。
- 2. **轨迹规划：**实现了分段五次多项式轨迹规划方法，通过中心差分法估算中间点速度和加速度，保证了关节运动的位置、速度、加速度连续性。
- 3. **基座切换：**创新性地实现了机械臂基座的动态切换机制，通过重新组织对象层级关系和关节映射，成功模拟了机械臂“换手行走”的运动模式。
- 4. **仿真验证：**在 CoppeliaSim 环境中完成了完整的运动仿真，机械臂从初始位置出发，经过位置 1，最终到达位置 2，顺利完成两步行走任务。